E-COMMERCE APPLICATION

Full-Stack Project Documentation

Spring Boot 3 + React + PostgreSQL + JWT + Docker

Milan — Februar 2026

1. PREGLED PROJEKTA

1.1 Šta je ovaj projekat?

Full-stack e-commerce aplikacija sa registracijom, loginom (JWT autentifikacija), CRUD operacijama za proizvode, korpom za kupovinu i narudžbinama. Backend je Spring Boot REST API, frontend je React SPA (Single Page Application), baza je PostgreSQL, sve pokrenuto u Docker-u.

1.2 Zašto baš ovaj projekat?

Pokriva SVE što firme pitaju na intervjuima za junior backend/full-stack pozicije:
- JWT autentifikacija i autorizacija (Spring Security)
- REST API dizajn sa svim HTTP metodama
- JPA/Hibernate sa relacijama (OneToMany, ManyToMany, ManyToOne)
- Validacija, error handling, DTO pattern
- React frontend sa zaštićenim rutama
- Docker i Docker Compose za deployment
- PostgreSQL baza podataka
- Git sa lepom istorijom commit-ova

1.3 Tehnologije

Sloj	Tehnologija	Verzija
Backend	Java	17
Backend	Spring Boot	3.2+
Backend	Spring Security	6+
Backend	Spring Data JPA	3.2+
Backend	JWT (jjwt library)	0.12+
Backend	Lombok	1.18+
Backend	Maven	3.9+
Baza	PostgreSQL	15+
Frontend	React	18+
Frontend	React Router	6+
Frontend	Axios	1.6+
DevOps	Docker	24+
DevOps	Docker Compose	2+

---

2. BAZA PODATAKA — ŠEMA

2.1 ER Dijagram (tabele i relacije)

Projekat ima 6 tabela:

Tabela: users

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
first_name	VARCHAR(50)	NOT NULL	Ime
last_name	VARCHAR(50)	NOT NULL	Prezime
email	VARCHAR(100)	UNIQUE, NOT NULL	Email (koristi se za login)
password	VARCHAR(255)	NOT NULL	BCrypt hash lozinke
role	VARCHAR(20)	NOT NULL, DEFAULT 'CUSTOMER'	ADMIN ili CUSTOMER
created_at	TIMESTAMP	NOT NULL	Datum registracije

Tabela: categories

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
name	VARCHAR(50)	UNIQUE, NOT NULL	Naziv kategorije (Elektronika, Obuća...)
description	VARCHAR(255)		Opis kategorije

Tabela: products

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
name	VARCHAR(100)	NOT NULL	Naziv proizvoda
description	TEXT		Opis proizvoda
price	DECIMAL(10,2)	NOT NULL	Cena
stock_quantity	INT	NOT NULL, DEFAULT 0	Količina na stanju
image_url	VARCHAR(255)		URL slike
category_id	BIGINT	FK → categories(id)	Kategorija proizvoda
created_at	TIMESTAMP	NOT NULL	Datum kreiranja

Tabela: orders

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
user_id	BIGINT	FK → users(id), NOT NULL	Ko je naručio
total_amount	DECIMAL(10,2)	NOT NULL	Ukupan iznos
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED
shipping_address	VARCHAR(255)	NOT NULL	Adresa dostave
created_at	TIMESTAMP	NOT NULL	Datum narudžbine

Tabela: order_items (stavke narudžbine)

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
order_id	BIGINT	FK → orders(id), NOT NULL	Koja narudžbina
product_id	BIGINT	FK → products(id), NOT NULL	Koji proizvod
quantity	INT	NOT NULL	Količina
price_at_purchase	DECIMAL(10,2)	NOT NULL	Cena u momentu kupovine

Tabela: cart_items (korpa)

Kolona	Tip	Ograničenja	Opis
id	BIGINT	PK, AUTO_INCREMENT	Primarni ključ
user_id	BIGINT	FK → users(id), NOT NULL	Čija korpa
product_id	BIGINT	FK → products(id), NOT NULL	Koji proizvod
quantity	INT	NOT NULL, DEFAULT 1	Količina

2.2 Relacije između tabela

users →1---N→ orders — Jedan korisnik ima više narudžbina (OneToMany)

orders →1---N→ order_items — Jedna narudžbina ima više stavki (OneToMany). Ovo je kao IznajmljivanjeStavkaIznajmljivanja iz biblioteke!

products →1---N→ order_items — Jedan proizvod može biti u više stavki (ManyToOne sa strane order_items)

categories →1---N→ products — Jedna kategorija ima više proizvoda (OneToMany)

users →1---N→ cart_items — Jedan korisnik ima više stavki u korpi (OneToMany)
products →1---N→ cart_items — Jedan proizvod može biti u više korpi (ManyToOne)

3. BACKEND ARHITEKTURA

3.1 Struktura paketa

```
com.milan.ecommerce/
  ├── config/           ← SecurityConfig, JwtAuthFilter, CorsConfig
  ├── controller/       ← AuthController, ProductController,
OrderController, CartController, CategoryController
  ├── dto/              ← RegisterRequest, LoginRequest, LoginResponse,
ProductDTO, OrderDTO, CartItemDTO
  ├── entity/           ← User, Product, Category, Order, OrderItem,
CartItem
  ├── enums/            ← Role (ADMIN, CUSTOMER), OrderStatus (PENDING,
CONFIRMED...)
  ├── exception/        ← GlobalExceptionHandler,
ResourceNotFoundException, DuplicateEmailException
  ├── repository/       ← UserRepository, ProductRepository,
OrderRepository, CartItemRepository, CategoryRepository
  ├── security/         ← JwtService, JwtAuthenticationFilter,
CustomUserDetailsService
  ├── service/          ← AuthService, ProductService, OrderService,
CartService, CategoryService
  └── EcommerceApplication.java
```

3.2 Tri sloja (isto kao u biblioteci!)

Controller → Service → Repository → Baza. Isti princip kao tvoj biblioteka projekat, samo sa Spring anotacijama umesto ručnog koda.

Controller (@RestController) — prima HTTP zahteve, vraća JSON. NE sadrži poslovnu logiku.

Service (@Service) — poslovna logika: provera da li korisnik postoji, preračunavanje cena, kreiranje JWT-a.

Repository (extends JpaRepository) — pristup bazi. Spring generiše SQL automatski.

4. AUTENTIFIKACIJA I AUTORIZACIJA (JWT + Spring Security)

Ovo je DEO KOJI TE KOŠTAO SYMPHONY DIGITAL-A. Nauči ga 100%.

4.1 Klase koje treba da napraviš

4.1.1 JwtService.java (kreira i validira tokene)

Metode:
- generateToken(UserDetails user) — kreira JWT sa emailom, ulogom i rokom trajanja
- extractEmail(String token) — čita email iz tokena
- isTokenValid(String token, UserDetails user) — proverava potpis i rok
- extractAllClaims(String token) — dekodira payload

SECRET_KEY čuvaš u application.yml (nikad hardkodirano u kodu!):

```
jwt:
  secret: tvoj-tajni-kljuc-barem-256-bita
  expiration: 86400000  # 24 sata u milisekundama
```

4.1.2 JwtAuthenticationFilter.java (proverava svaki zahtev)

extends OncePerRequestFilter. Za SVAKI HTTP zahtev:

1. Izvuci Authorization header
2. Ako počinje sa 'Bearer ' — izvuci token
3. Pozovi JwtService.extractEmail(token) — dobij email
4. Pozovi JwtService.isTokenValid() — proveri potpis i rok
5. Ako je validan — postavi korisnika u SecurityContext (Spring sad zna ko je ulogovan)
6. Ako nije validan ili nema token — pusti dalje (Spring Security će blokirati ako endpoint zahteva autentifikaciju)

4.1.3 SecurityConfig.java (pravila pristupa)

@Configuration klasa koja definiše:

```
http.authorizeHttpRequests(auth -> auth
  .requestMatchers("/api/auth/**").permitAll()          // login i register
— svi
  .requestMatchers(GET, "/api/products/**").permitAll()  // pregled
proizvoda — svi
  .requestMatchers(GET, "/api/categories/**").permitAll()
  .requestMatchers("/api/admin/**").hasRole("ADMIN")     // admin endpointi
  .anyRequest().authenticated()                          // sve ostalo —
mora biti ulogovan
);
```

Takođe registruje JwtAuthenticationFilter PRE UsernamePasswordAuthenticationFilter-a.

4.1.4 CustomUserDetailsService.java

implements UserDetailsService. Metoda loadUserByUsername(email) — traži korisnika u bazi po emailu. Spring Security koristi ovu klasu da proveri kredencijale pri loginu.

4.2 Login tok — korak po korak

1. Klijent šalje POST /api/auth/login sa { email, password }
2. AuthController prima, poziva AuthService.login()
3. AuthService koristi AuthenticationManager.authenticate() — Spring Security proverava lozinku
4. Ako je tačna — AuthService poziva JwtService.generateToken(user)
5. Token se vraća klijentu u LoginResponse DTO: { token: 'eyJhbG...', email: 'marko@gmail.com', role: 'CUSTOMER' }
6. Klijent čuva token (localStorage ili state u React-u)

4.3 Zaštićen zahtev — korak po korak

1. Klijent šalje GET /api/cart sa headerom Authorization: Bearer eyJhbG...
2. JwtAuthenticationFilter presreće zahtev PRE Controller-a
3. Izvlači token, proverava potpis, čita email i ulogu
4. Postavlja korisnika u SecurityContext
5. Spring Security proverava pravila iz SecurityConfig — da li uloga ima pristup
6. Ako DA — zahtev stiže do CartController-a
7. Ako NE — vraća 401 Unauthorized ili 403 Forbidden

5. ENTITETI (JPA klase)

5.1 User.java

```
@Entity
@Table(name = "users")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class User implements UserDetails {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String firstName;
    private String lastName;
    @Column(unique = true, nullable = false)
    private String email;
    private String password;
    @Enumerated(EnumType.STRING)
    private Role role;
    @CreationTimestamp
    private LocalDateTime createdAt;
    @OneToMany(mappedBy = "user")
    private List<Order> orders;
    // implements UserDetails metode za Spring Security
}
```

BITNO: User implements UserDetails — Spring Security zahteva ovo za autentifikaciju. Moraš implementirati metode: getAuthorities() (vraća ulogu), getUsername() (vraća email), isAccountNonExpired(), isEnabled() itd.

5.2 Product.java

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @Column(columnDefinition = "TEXT")
    private String description;
    private BigDecimal price;
    private Integer stockQuantity;
    private String imageUrl;
    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

5.3 Order.java

```
@Entity
```

```java
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;
    private BigDecimal totalAmount;
    @Enumerated(EnumType.STRING)
    private OrderStatus status;
    private String shippingAddress;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderItem> items;
    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

BITNO: cascade = CascadeType.ALL znači da kad sačuvaš Order, automatski se čuvaju i OrderItem-i. Isto kao tvoj KreirajIznajmljivanjeSO, samo automatski!

5.4 OrderItem.java

```java
@Entity
public class OrderItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;
    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;
    private Integer quantity;
    private BigDecimal priceAtPurchase;
}
```

priceAtPurchase čuva cenu u momentu kupovine. Ako se cena proizvoda promeni sutra, tvoja narudžbina zadržava staru cenu.

5.5 Category.java i CartItem.java

Category: id, name, description, @OneToMany(mappedBy='category') List<Product> products.

CartItem: id, @ManyToOne User user, @ManyToOne Product product, Integer quantity.

6. REST API ENDPOINTI

6.1 Autentifikacija (/api/auth) — OTVORENO ZA SVE

Metoda	Endpoint	Opis	Body/Odgovor
POST	/api/auth/register	Registracija novog korisnika	Body: {firstName, lastName, email, password}. Odgovor: 201 Created
POST	/api/auth/login	Login — vrati JWT token	Body: {email, password}. Odgovor: {token, email, role}

6.2 Proizvodi (/api/products) — GET otvoreno, ostalo ADMIN

Metoda	Endpoint	Pristup	Opis
GET	/api/products	Svi	Svi proizvodi (sa paginacijom i pretragom)
GET	/api/products/{id}	Svi	Jedan proizvod po ID-ju
GET	/api/products/category/{categoryId}	Svi	Proizvodi po kategoriji
POST	/api/products	ADMIN	Dodaj novi proizvod
PUT	/api/products/{id}	ADMIN	Izmeni proizvod
DELETE	/api/products/{id}	ADMIN	Obriši proizvod

6.3 Kategorije (/api/categories) — GET otvoreno, ostalo ADMIN

Metoda	Endpoint	Pristup	Opis
GET	/api/categories	Svi	Sve kategorije
POST	/api/categories	ADMIN	Dodaj kategoriju
PUT	/api/categories/{id}	ADMIN	Izmeni kategoriju
DELETE	/api/categories/{id}	ADMIN	Obriši kategoriju

6.4 Korpa (/api/cart) — SAMO ULOGOVANI

Metoda	Endpoint	Opis
GET	/api/cart	Prikaži moju korpu (sve stavke)
POST	/api/cart	Dodaj proizvod u korpu. Body: {productId, quantity}

PUT	/api/cart/{cartItemId}	Izmeni količinu. Body: {quantity}
DELETE	/api/cart/{cartItemId}	Ukloni stavku iz korpe
DELETE	/api/cart	Isprazni celu korpu

BITNO: Server zna KO je ulogovan iz JWT tokena (SecurityContext). Ne šalješ userId — server ga sam izvlači!

6.5 Narudžbine (/api/orders) — ULOGOVANI + ADMIN

Metoda	Endpoint	Pristup	Opis
POST	/api/orders	CUSTOMER	Kreiraj narudžbinu iz korpe. Body: {shippingAddress}
GET	/api/orders	CUSTOMER	Moje narudžbine
GET	/api/orders/{id}	CUSTOMER	Detalji jedne narudžbine
GET	/api/admin/orders	ADMIN	SVE narudžbine svih korisnika
PUT	/api/admin/orders/{id}/status	ADMIN	Promeni status. Body: {status: 'SHIPPED'}

6.6 Kreiranje narudžbine — detaljan tok

Ovo je NAJKOMPLIKOVANIJI deo (kao KreirajIznajmljivanjeSO u biblioteci):

1. Korisnik pozove POST /api/orders sa { shippingAddress: 'Kneza Milosa 5' }
2. OrderService uzme korisnika iz SecurityContext-a
3. Uzme sve CartItem-e tog korisnika iz baze
4. Za svaki CartItem kreira OrderItem sa priceAtPurchase = product.getPrice()
5. Izračuna totalAmount = suma svih (quantity * priceAtPurchase)
6. Proveri da li ima dovoljno na stanju (stockQuantity >= quantity)
7. Smanji stockQuantity za svaki proizvod
8. Sačuva Order sa svim OrderItem-ima (cascade = ALL — jedan save())
9. Obriši sve CartItem-e tog korisnika (korpa se isprazni posle kupovine)
10. Sve pod @Transactional — ako bilo šta pukne, rollback

7. DTO PATTERN (Data Transfer Object)

NIKAD ne vraćaj Entity direktno klijentu! Korisnik ne treba da vidi password hash ili interne ID-jeve. Koristi DTO — objekat samo sa poljima koja klijent treba da vidi.

RegisterRequest.java

```
public record RegisterRequest(
    @NotBlank String firstName,
    @NotBlank String lastName,
    @Email @NotBlank String email,
    @Size(min = 6) String password
) {}
```

LoginRequest.java

```
public record LoginRequest(
    @Email @NotBlank String email,
    @NotBlank String password
) {}
```

LoginResponse.java

```
public record LoginResponse(String token, String email, String role) {}
```

ProductDTO.java

```
public record ProductDTO(Long id, String name, String description,
    BigDecimal price, Integer stockQuantity, String imageUrl,
    String categoryName) {}
```

Vidiš: umesto Category objekta, vraćaš samo categoryName (String). Klijent ne treba ceo objekat.

8. ERROR HANDLING

Napravi GlobalExceptionHandler sa @RestControllerAdvice koji hvata sve exception-e i vraća čist JSON odgovor:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(404).body(new ErrorResponse(404,
ex.getMessage()));
    }
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<ErrorResponse>
handleDuplicate(DuplicateEmailException ex) {
        return ResponseEntity.status(409).body(new ErrorResponse(409,
ex.getMessage()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(...) { ... }
}
```

Umesto da klijent dobije 500 Internal Server Error sa stack trace-om, dobija čist JSON: { "status": 404, "message": "Product not found with id: 5" }

9. REACT FRONTEND

9.1 Struktura

```
src/
  ├── components/       ← Navbar, ProductCard, CartItem, ProtectedRoute
  ├── pages/            ← HomePage, LoginPage, RegisterPage, ProductsPage,
CartPage, OrdersPage, AdminDashboard
  ├── services/         ← api.js (Axios instance sa JWT), authService,
productService, cartService, orderService
  ├── context/          ← AuthContext.jsx (drži info o ulogovanom
korisniku)
  ├── App.jsx           ← React Router rute
  └── main.jsx
```

9.2 Stranice

Stranica	Ruta	Pristup	Opis
HomePage	/	Svi	Hero sekcija + featured proizvodi
ProductsPage	/products	Svi	Lista proizvoda sa pretragom i filterom po kategoriji
ProductDetailPage	/products/:id	Svi	Detalji proizvoda + dugme 'Dodaj u korpu'
LoginPage	/login	Svi	Login forma → dobija token → čuva u context
RegisterPage	/register	Svi	Registracija forma
CartPage	/cart	Ulogovani	Korpa sa stavkama + 'Naruči' dugme
OrdersPage	/orders	Ulogovani	Moje narudžbine sa statusom
AdminDashboard	/admin	ADMIN	CRUD za proizvode i kategorije + sve narudžbine

9.3 JWT u React-u

api.js — Axios instance sa interceptorom koji automatski dodaje JWT token u svaki zahtev:

```
const api = axios.create({ baseURL: 'http://localhost:8080/api' });
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

ProtectedRoute komponenta — proverava da li je korisnik ulogovan pre nego prikaže stranicu. Ako nije — redirect na /login.

10. DOCKER I DOCKER COMPOSE

10.1 Dockerfile (backend)

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/ecommerce-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

10.2 docker-compose.yml

```
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: ecommerce
      POSTGRES_USER: milan
      POSTGRES_PASSWORD: password123
    ports:
      - '5432:5432'
  backend:
    build: ./backend
    ports:
      - '8080:8080'
    depends_on:
      - postgres
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/ecommerce
  frontend:
    build: ./frontend
    ports:
      - '3000:3000'
```

Jedna komanda docker-compose up pokreće celu aplikaciju: bazu, backend i frontend.

11. REDOSLED IMPLEMENTACIJE — KORAK PO KORAK

Radi OVIM REDOM. Svaki korak završi i testraj pre nego što pređeš na sledeći.

Dan 1-2: Inicijalizacija + Entiteti

1. Kreiraj Spring Boot projekat na start.spring.io (dependencies: Web, JPA, Security, PostgreSQL, Lombok, Validation)
2. Podesi application.yml (baza, JPA, server port)
3. Napravi sve entitete: User, Product, Category, Order, OrderItem, CartItem
4. Napravi enum-e: Role, OrderStatus
5. Pokreni aplikaciju — Hibernate treba da kreira tabele automatski (spring.jpa.hibernate.ddl-auto=create)

Dan 3-4: Security + Auth

6. Napravi JwtService (generateToken, extractEmail, isTokenValid)
7. Napravi CustomUserDetailsService (loadUserByUsername)
8. Napravi JwtAuthenticationFilter (extends OncePerRequestFilter)
9. Napravi SecurityConfig (pravila pristupa, registracija filtera, PasswordEncoder bean)
10. Napravi AuthController + AuthService (register i login endpointi)
11. Testiraj sa Postman-om: register korisnika, login, dobij token, šalji token u headeru

Dan 5-6: CRUD Product + Category

12. Napravi CategoryRepository, CategoryService, CategoryController (CRUD)
13. Napravi ProductRepository, ProductService, ProductController (CRUD)
14. Napravi DTO-ove: ProductDTO, CategoryDTO
15. Napravi GlobalExceptionHandler
16. Testiraj: dodaj kategorije, dodaj proizvode, pretraži, izmeni, obriši

Dan 7-8: Cart + Orders

17. Napravi CartItemRepository, CartService, CartController
18. Napravi OrderRepository, OrderItemRepository, OrderService, OrderController
19. Implementiraj kreiranje narudžbine iz korpe (najkomplikovaniji deo — korak 6.6)
20. Admin endpointi: sve narudžbine, promena statusa
21. Testiraj ceo tok: login → dodaj u korpu → naruči → proveri narudžbinu

Dan 9-10: React frontend

22. npx create-react-app frontend (ili Vite)
23. Instaliraj: axios, react-router-dom
24. Napravi api.js sa Axios interceptorom za JWT
25. Napravi AuthContext za čuvanje tokena i korisnika

26. Napravi LoginPage i RegisterPage
27. Napravi ProductsPage (lista proizvoda sa karticama)
28. Napravi CartPage i OrdersPage
29. Napravi ProtectedRoute komponentu
30. Napravi Navbar sa login/logout i linkovima

Dan 11-12: Docker + GitHub + Poliranje

31. Napravi Dockerfile za backend
32. Napravi docker-compose.yml
33. Testiraj sa docker-compose up
34. Napravi lep README.md na GitHub-u sa: opisom projekta, tehnologijama, screenshot-ovima, uputstvom za pokretanje
35. Očisti kod, dodaj komentare, proveri da SVE radi

12. KAKO BRANITI OVAJ PROJEKAT NA INTERVJUU

Pitanje: Kako radi autentifikacija?

"Koristim JWT token sa Spring Security. Korisnik se registruje — lozinka se hashira sa BCrypt i čuva u bazi. Pri loginu, server proveri kredencijale, kreira JWT sa emailom, ulogom i rokom trajanja, i vrati ga klijentu. Klijent šalje token u Authorization headeru svakog zahteva. JwtAuthenticationFilter presreće zahtev, validira potpis tokena bez pristupa bazi, čita ulogu i postavlja korisnika u SecurityContext. SecurityConfig definiše koja uloga sme na koji endpoint."

Pitanje: Kako radi kreiranje narudžbine?

"Korisnik dodaje proizvode u korpu (cart_items tabela). Kad klikne Naruči, server uzme sve stavke iz korpe, za svaku kreira OrderItem sa cenom u momentu kupovine (priceAtPurchase), izračuna ukupan iznos, smanji stock_quantity za svaki proizvod, sačuva Order sa CascadeType.ALL (automatski se čuvaju i OrderItem-i), i obriše korpu. Sve pod @Transactional — ako bilo šta pukne, rollback."

Pitanje: Zašto DTO a ne Entity?

"Nikad ne vraćam Entity direktno jer: (1) password hash ne sme do klijenta, (2) cirkularni JSON — User ima orders, Order ima user, beskonačna petlja, (3) fleksibilnost — mogu da menjam bazu bez da menjam API odgovor. DTO sadrži samo polja koja klijent treba da vidi."

Pitanje: Kako bi dodao novu funkcionalnost, npr. wishlist?

"Kreirao bih WishlistItem entitet sa @ManyToOne User i @ManyToOne Product. Napravio bih WishlistRepository (extends JpaRepository), WishlistService sa poslovnom logikom, i WishlistController sa CRUD endpointima. U SecurityConfig bih dodao /api/wishlist/** za autentifikovane korisnike. Na frontu bih dodao WishlistPage i servis."

Pitanje: Šta je CascadeType.ALL?

"Znači da kad sačuvam Order, automatski se čuvaju i svi OrderItem-i bez da pozivam save() za svaki posebno. Isto važi za delete — obrisanje Order-a automatski briše stavke. U mom biblioteka projektu sam morao ručno da prolazim kroz listu stavki i pozivam broker.add() za svaku. Sa JPA i cascade to je automatski."